

Lorem ipsum

Grobe Struktur:

1 Einleitung

Transformer-Architekturen haben sich in den letzten Jahren vor allem im Natural-Language-Processing bewährt, dabei ist diese Modell-Klasse vielseitig einsetzbar [Hier Zitate von anderen Domänen]. Auch für unser Problem, der Generierung einer Sequence, kann eine Transformer-Architektur genutzt werden.

Transformer-Modell seit ChatGPT sehr bewährt, blibliablu

2 Theorie

[Gleichheiten zu RNNs]. Auch hier wollen wir $N = 128$ Noten als Input liefern und die nächsten $O = 1$ vorhersagen lassen. Die Folge $(x_1, x_2, \dots, x_{N+1})$ geben wir dann wieder in das Modell und erhalten so schließlich das generierte Stück. Wie auch im Fall der RNNs ist unser Preprocessing denkbar einfach, wieder 88 Noten. Nur fügen wir nun noch eine SOS- und EOS-Token ein (die Abkürzungen stehen für Start-of-Sequence bzw. End-of-Sequence). Unser "Vokabular" hat somit Größe 90; der SOS-Token dient dazu, dass Modell später überhaupt zum Generieren zu bringen. Der EOS-Token bietet den Vorteil, dass das Modell selbst entscheidet, wann das Stück fertig komponiert ist, so kann das Modell ggf. bestimmte Schluss-Kadenzen und -muster erkennen und übernehmen.

3 Preprocessing

Genau gleich wie bei KK

4 Auch hier ein erster Versuch

5 Weitere Versuche

6 Ein etwas anderes Preprocessing

7 Analog zu RNNs

Seq -i [0, 127] fruchtet nicht, max. Acc nach 20 Epochen bei ca. 65prz
Wieso, weshalb, warum?

8 Ein neuer Ansatz

Tokenization der folgenden Form: $(n, v) \mapsto n + (v - 1) \cdot 128$, n: note, v: voice/Stimm Lage (i.e. Bass = 1, Sopran etc.), führt zu 514 vocab size (inkl. EOS, PAD tokens)

Bereits kleine Architektur mit 130k Parametern erreicht ca. 84prz acc, wunderlicherweise sogar nach der ersten -zweiten Epoche. Ab dann plateaued aber die Acc, wieso, weshalb, warum?

9 Größere Architektur

Was verbessert sich im Vergleich zu oben

10 Benchmarks

- Acc - Confidence vs. Seq-Länge

11 Notation

$\ell_{\text{SEQ}} = 512$: Maximale Sequenzlänge (SEQ_LEN)
 d_{embed} : Dimension der Embeddings (EMBED_DIM)
 d_{ffw} : Dimension des Feed-Forward-Netzwerkes (FFW_DIM)
 N_H : Anzahl der Attention-Heads (NUM_HEADS)

12 Musikgenerierung mit Transformern

12.1 Preprocessing

Für die Transformer-Architektur passen wir das Preprocessing ein wenig an. Jeden Choral erweitern wir um einen SOS-Token (Start-Of-Sequence) am Anfang und einen EOS-Token (End-Of-Sequence) am Ende. Alle diese modifizierten Choräle konkatenieren wir dann zu einem einzigen Vektor D .

Sei $C_i = [n_{i,1}, \dots, n_{i,\ell}]$ der i -ten Choral, der Datenvektor D ergibt sich dann als

$$D = [\text{SOS}, \underbrace{n_{1,1}, \dots, n_{1,\ell_1}}_{C_1}, \text{EOS}, \text{SOS}, \underbrace{n_{2,1}, \dots, n_{2,\ell_2}}_{C_2}, \text{EOS}, \dots, \text{EOS}]$$

Wir schreiben $D = [t_1, \dots, t_n]$ (wobei $t_1 = \text{SOS}$, $t_2 = n_{1,1}$ usw.), die einzelnen t s bezeichnen wir als Tokens.

Ein Input-Output-Paar (x, y) ist definiert als:

$$\begin{aligned} x &= [t_k, \dots, t_{k+\ell_{\text{SEQ}}-1}] \\ y &= [t_{k+1}, \dots, t_{k+\ell_{\text{SEQ}}}] \end{aligned}$$

Das Target y ist also die um eine Position verschobene Input-Sequenz.

Unter Berücksichtigung der drei Spezialtokens SOS, EOS und PAD erhalten wir eine Vokabulargröße V von 131.

12.2 Musikgenerierung mit Transformern

Der Generierungsprozess verläuft analog zu RNNs durch Sampling aus der Wahrscheinlichkeitsverteilung des Modells.

In unserem Fall gestalten wir die Generierung mit Transformern etwas flexibler: An die Anfangssequenz wird nur noch folgende Anforderung gestellt $[n_1, \dots, n_k]$, mit $0 \leq k \leq \ell_{\text{SEQ}}$ (wobei wir für $k = 0$ eine leere Liste meinen), das ist Konsequenz des SOS-Tokens. Wie sich die Länge der Anfangssequenz auf die Vorhersage des Modells auswirkt, untersuchen wir in subsection 12.6.

Ebenso lassen wir das Modell "entscheiden", wann das generierte Stück endet; die Konsequenz des EOS-Tokens (um Endlosschleifen zu vermeiden, ist eine maximale Länge aber dennoch zu setzen).

Die Audiodateien finden sich unter folgenden Link: [HIER LINK EINFÜGEN].

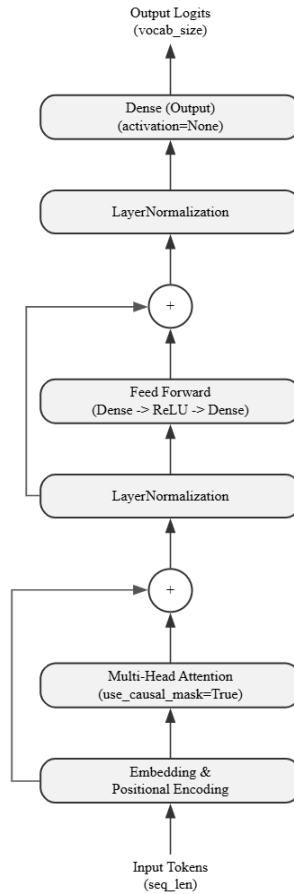


Figure 1: Grundlegende Struktur unseres Transformers

12.3 Die grundlegende Transformer-Architektur

Für die konkrete Struktur orientieren wir uns an Chollet, 2021, Da wir während der Inferenz nur Zugriff auf die bisherigen Noten haben, implementieren wir einen Decoder-Only-Transformer. Die in Figure 1 dargestellte Architektur lässt sich in folgende Komponenten unterteilen, wobei wir an entscheidenden Stellen Hyperparameter wählen müssen, die die Kapazität und das Lernverhalten des Modells beeinflussen:

- Wie im Ansatz mit RNNs werden die diskreten Input-Tokens in Vektoren der Dimension d_{emb} eingebettet. Um Informationen über die Positionen zu erhalten, addieren wir positionsabhängige Vektoren gleicher Dimension, in unserem Fall nutzen wir das in Vaswani et al., 2017 vorgestellte Positional-Encoding.

- Das Herzstück unseres Transformers ist natürlich die Multi-Head-Attention, die Anzahl der Attentions-Heads N_H können wir als weiteren Hyperparameter wählen. Entscheidend ist für uns vorallem die Maskierung: Da wir autoregressiv generieren, darf das Modell an Position t nicht keine Informationen der noch zu generierenden Noten an $t + 1, \dots, \ell_{\text{SEQ}}$ erhalten.
- Nach der Attention-Schicht folgt ein einfaches Feed-Forward-Netzwerk, in unserem Fall bestehend aus zwei Schichten. Die Dimension innerhalb dieses Netzwerkes ist ein weiterer Hyperparameter d_{ffw} .
- Schließlich bildet die letzte Schicht auf einen Vektor der Dimension V ab, wir nutzen keine Aktivierungsfunktion, erhalten die sog. Logits als Output. Wendet man auf diese die Softmax-Funktion an, erhält man die Wahrscheinlichkeitsverteilung für das Noten-Sampling.

12.4 Ein erster Versuch

Nun stehen wir nicht mehr ganz am Anfang, wir recyceln für den ersten Versuch die "naive" Idee, die im Falle von RNNs erfolgreich war; unser Modell soll also **eine** Note aus $[0, 127]_{\mathbb{Z}}$ vorhersagen.

Gem. subsection 12.3 haben wir 3 Hyperparameter zu bestimmen, wir wählen diese analog zum RNN, siehe Figure 2:

- $d_{\text{embed}} = 64$
- $N_H = 4$
- $d_{\text{ffw}} = 128$

Wir trainieren ebenfalls für 20 Epochen mit EarlyStopping und ReduceLROnPlateau. Nach 15 Epochen wird das Training beendet und das beste Modell, Epoche 10, übernommen: val-accuracy von ca. 84% und test-accuracy von ca. 82%.

12.5 Vom Bächlein zum Bach

Auch mit Transformern kommen wir scheinbar nicht über 85% Validation-Accuracy hinaus.

12.5.1 Data-Augmentation

Der Trainings-Korpus besteht gerade einmal aus etwas mehr als 220 000 Tokens.

12.5.2 Größere Modelle

12.6 Benchmarking

Layer (type)	Output Shape	Param #	Connected to
token_input (InputLayer)	(None, 512)	0	-
embedding (Embedding)	(None, 512, 64)	8,384	token_input[0][0]
add_6 (Add)	(None, 512, 64)	0	embedding[0][0]
attention (MultiHeadAttentio...	(None, 512, 64)	66,368	add_6[0][0], add_6[0][0]
add_7 (Add)	(None, 512, 64)	0	add_6[0][0], attention[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 512, 64)	128	add_7[0][0]
sequential_2 (Sequential)	(None, 512, 64)	16,576	layer_normalizat...
add_8 (Add)	(None, 512, 64)	0	layer_normalizat... sequential_2[0][...
layer_normalizatio... (LayerNormalizatio...	(None, 512, 64)	128	add_8[0][0]
logits (Dense)	(None, 512, 131)	8,515	layer_normalizat...

Total params: 100,099 (391.01 KB)

Trainable params: 100,099 (391.01 KB)

Non-trainable params: 0 (0.00 B)

Figure 2: Modell-Zusammenfassung mit $d_{\text{embed}} = 64$, $N_H = 4$ und $d_{\text{ffw}} = 128$